Submitted for publication

Finding the Maximum Area Axis-Parallel Rectangle in a Polygon*

Karen Daniels † *Victor Milenkovic*‡ *Dan Roth*§

Harvard University Division of Applied Sciences,

Center for Research in Computing Technology,

Cambridge, MA 02138.

October 1993

*An earlier version of this paper appeared in the proceedings of the Fifth Annual Canadian Conference on Computational Geometry, 1993.

†This research was funded by the Textile/Clothing Technology Corporation from funds awarded to them by the Alfred P. Sloan Foundation.

‡This research was funded by the Textile/Clothing Technology Corporation from funds awarded to them by the Alfred P. Sloan Foundation and by NSF grants CCR-91-157993 and CCR-90-09272.

§Supported by NSF grants CCR-89-02500 and CCR-92-00884 and by DARPA AFOSR-F4962-92-J-0466.

Running title: Finding the Maximum Area Axis-Parallel Rectangle in a Polygon

Contact author:

Karen Daniels ,

Aiken Computation Laboratory,

Harvard University,

Cambridge, MA 02138.

Abstract

We consider the geometric optimization problem of finding the maximum area axis-parallel rectangle (MAAPR) in an n-vertex general polygon. We characterize the MAAPR for general polygons by considering different cases based on the types of contacts between the rectangle and the polygon. We present a general framework for solving a key subcase of the MAAPR problem which dominates the running time for a variety of polygon types. Using this framework, we obtain the following MAAPR time results: $\Theta(n)$ for xy-monotone polygons, $O(n\alpha(n))$ for orthogonally convex polygons, $O(n\alpha(n)\log n)$ for horizontally (vertically) convex polygons, (where $\alpha(n)$ is the slowly growing inverse of Ackermann's function), $O(n\log n)$ for a special type of horizontally convex polygon, and $O(n\log^2 n)$ for general polygons. For all these types of non-rectilinear polygons, we match the running time of the best known algorithms for their rectilinear counterparts.

We prove a lower bound of time in $\Omega(n\log n)$ for finding the MAAPR in both self-intersecting polygons and general polygons with holes. The latter result gives us both a lower bound of $\Omega(n\log n)$ and an upper bound of $O(n\log^2 n)$ for general polygons with holes.

Keywords: Rectangles, geometric optimization

1 Introduction

The problem of finding the maximum area axis-parallel rectangle (MAAPR) inside a general polygon[1] of n vertices is a geometric optimization problem in the class of polygon *inclusion* problems [10]: $Inc(\mathcal{P}, \mathcal{Q}, \mu)$: Given $P \in \mathcal{P}$, find the μ-largest $Q \in \mathcal{Q}$ inside P, where $\mathcal{P}$ and $\mathcal{Q}$ are families of polygons, and μ is a real function on polygons such that:

$$\forall P \in \mathcal{P}, Q \in \mathcal{Q}, P \subseteq Q \Rightarrow \mu(P) \leq \mu(Q)$$

Our problem is an inclusion problem where $\mathcal{Q}$ is the set of axis-parallel rectangles, $\mathcal{P}$ is the set of polygons, and μ is the area function.

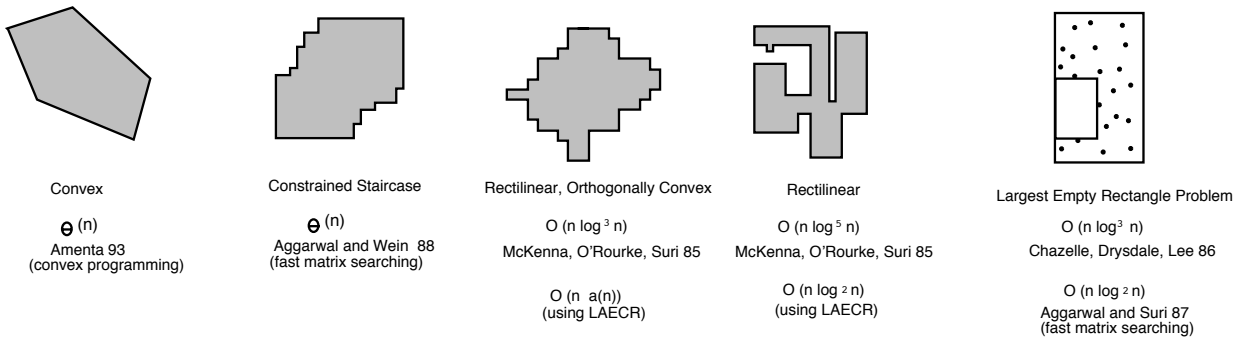


Figure 1: Related Work

This rectangle problem arises naturally in applications where a quick internal approximation to a polygon is useful. It is needed, for example, in the industrial problem of laying out apparel pattern pieces on clothing "markers" with minimal cloth waste [14, 15] (see Section 7). Despite its practical importance, work on finding the MAAPR has been restricted to rectilinear polygons [5, 13, 20] and, recently, convex polygons [6] (see Figure 1). Amenta [6] has shown that the MAAPR in a convex polygon can be found in linear time by phrasing it as a convex programming problem. For a constrained type of rectilinear polygon, Aggarwal and Wein [5] give a $\Theta(n)$ time algorithm for finding the MAAPR using the monotonicity of an area matrix associated with the polygon.

McKenna et.al. [13] use a divide-and-conquer approach to find the MAAPR in a rectilinear polygon in $O(n \log^5 n)$ time. For the merge step at the first level of divide-and-conquer, they obtain a

[1] A general polygon is a polygonal region in the plane with n vertices and an arbitrary number of components and holes.

vertically separated, horizontally convex rectilinear polygon[2]. At the second level, their merge step produces a rectilinear orthogonally convex[3] polygon, for which they solve the problem in $O(n \log^3 n)$ time. They also establish a lower bound of time in $\Omega(n \log n)$ for finding the MAAPR in rectilinear polygons with degenerate holes, which implies a lower bound for general polygons with degenerate holes.

McKenna et.al. note, without giving details, that the MAAPR in a rectilinear polygon can also be found using the more complicated $O(n \log^3 n)$ time divide-and-conquer algorithm of Chazelle, et. al.[8] for the largest empty rectangle (LER) problem. LER is stated as follows: given a rectangle containing a set S of n points, find the largest area subrectangle with sides parallel to those of the original rectangle which contains no points from S ([8, 16, 3]). Chazelle et. al. observe that the running time of the merge step of their algorithm is dominated by the largest-area empty corner rectangle (LAECR) problem: given two subsets L and R of S, find the largest rectangle containing no point of S which has lower-left corner in L and upper-right corner in R. The fastest solution to LAECR is Aggarwal and Suri's $O(n \log n)$ time algorithm, which they present as part of an $O(n \log^2 n)$ time solution to LER [3]. This LAECR algorithm relies on fast searching of area matrices.

We observe that a speed-up in the LAECR algorithm automatically improves the running time for finding the MAAPR in a rectilinear polygon. (Other relationships may well exist between the LER and the rectilinear polygon problem.) The speed-up is possible because the dominant task for finding the MAAPR in a vertically separated, horizontally convex rectilinear polygon can be accomlished using the LAECR algorithm. This yields $O(n \log^2 n)$ time for finding the MAAPR in a rectilinear polygon.

The $O(n \log^3 n)$ time algorithm of [13] for rectilinear orthogonally convex polygons can also be improved by applying recent results in fast matrix searching. Aggarwal and Suri [3] note that, for the LAECR problem which can be associated with the vertices of this type of polygon, there is a corresponding area matrix whose maximum can be found in $O(n \log n)$ time by decomposing it

[2]A vertically separated, horizontally convex polygon has left and right chains (extending from highest to lowest point of the polygon) on opposite sides of some vertical line.

[3]An orthogonally convex polygon is both horizontally and vertically convex. This class contains the class of convex polygons.

into a set of simpler area matrices. They note in [4] that Klawe and Kleitman's results [12] for this simpler type of matrix imply $O(n\alpha(n))$ time for the more complex matrix, where $\alpha(n)$ is the slowly growing inverse of Ackermann's function. This, we observe, yields $O(n\alpha(n))$ time for the rectilinear orthogonally convex polygon.

No algorithm is known for finding the MAAPR in a general polygon with (non-degenerate) holes, nor has a lower bound tighter than $\Omega(n)$ been established. We present the first algorithmic results for the general polygon with holes: an $O(n \log^2 n)$ time algorithm. We also prove a lower time bound for this type of polygon of $\Omega(n \log n)$. The divide-and-conquer approach used in the rectilinear case is applicable to non-rectilinear polygons, but dealing with the low-level non-rectilinear polygons is a challenge. As is the case for rectilinear polygons, the running time is dominated by the equivalent of the LAECR problem. For this reason, we present a general framework which we use to transform the LAECR problem for several types of non-rectilinear polygons into LAECR problems for "almost rectilinear" polygons. This allows us to achieve the same MAAPR time bounds for each non-rectilinear case as has already been achieved for the corresponding rectilinear case.

Our paper is organized as follows. In Section 2 we characterize the MAAPR for general polygons by considering different cases based on the types of contacts between the rectangle and the polygon. In Section 3 we present a general framework for solving the 2-contact case of the MAAPR problem, which dominates the running time for a variety of polygon types. The framework involves transforming the polygon, via vertex projection and inner rectilinear approximations, into another polygon for which we can solve the associated LAECR problem. Within this framework, we show that the 2-contact case can be solved in $\Theta(n)$ time for an xy-monotone polygon[4], in $O(n\alpha(n))$ time for an orthogonally convex polygon, and in $O(n \log n)$ time for a vertically separated horizontally convex polygon.

In Section 4 we summarize the results of [2, 12, 1, 3] relating to efficient solutions for all of the monotone matrix searching problems associated with the LAECR algorithms of Section 3. This leads to the algorithmic results of Section 5. First, we present a $\Theta(n)$ time result for xy-monotone polygons and an $O(n\alpha(n))$ time algorithm for orthogonally convex polygons. Then, the orthogonally convex polygon result is used as the basis for an $O(n\alpha(n) \log n)$ time divide-and-conquer algorithm for

[4]A simple polygon consisting of two xy-monotone chains is an xy-monotone polygon.

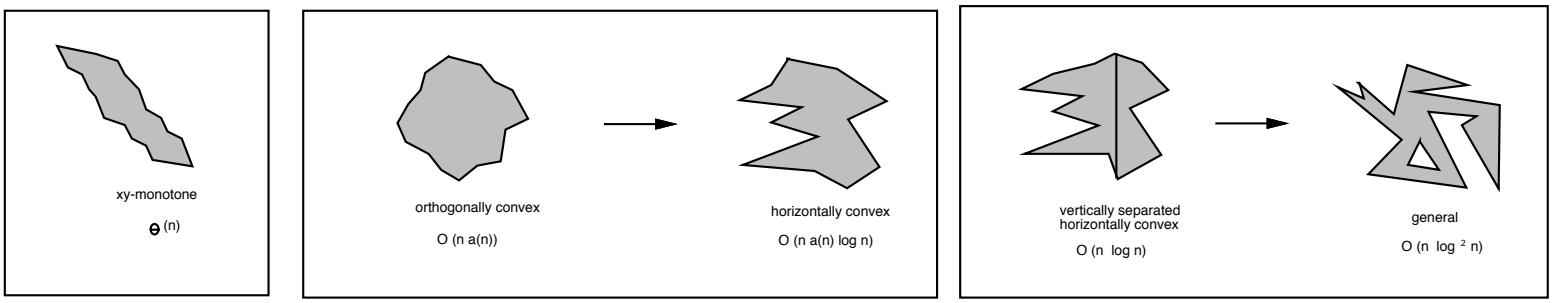


Figure 2: Results for Different Types of Polygons

horizontally (vertically) convex polygons. If used in the divide-and-conquer algorithm of McKenna et. al. [13], this immediately gives $O(n\alpha(n)\log^2 n)$ time for general polygons. However, we give a faster $O(n\log^2 n)$ divide-and-conquer algorithm rfor general polygons based on an $O(n\log n)$ time algorithm for the vertically separated, horizontally convex polygon. The $O(n\log n)$ algorithm uses the results of Aggarwal and Suri [3, 4] for LAECR. Figure 2 summarizes our algorithmic results.

In Section 6 we prove a lower bound of time in $\Omega(n\log n)$ for finding the MAAPR in both self-intersecting polygons and general polygons with holes. The latter result gives us both a lower bound of $\Omega(n\log n)$ and an upper bound of $O(n\log^2 n)$ for general polygons with holes. It uses symbolic perturbation to extend the $\Omega(n\log n)$ lower bound of McKenna et. al. for rectilinear polygons with degenerate holes. The proof for self-intersecting polygons involves a reduction from MAX-GAP. These lower bounds contrast with the $\Theta(n)$ results achievable for the corresponding enclosure problems.

Section 7 discusses MAAPR applications.

2 Characterizing the MAAPR

In this section we characterize the MAAPR contained in a general polygon P by considering different cases based on the types of contacts between the MAAPR and the boundary of P, and outline a naive algorithm for finding the MAAPR based on this characterization.

2.1 Types of Contacts

Intuitively, if an axis-parallel rectangle is inside P, it has four degrees of freedom (parameters) and can "grow" until each of its four sides is stopped by contact with the boundary of P. Contacts between the MAAPR and P are of two types: 1) a side of the MAAPR with a vertex of P, and 2) a corner of the MAAPR with an edge of P. In order to discuss the first type, we require the notion of a *reflex extreme* vertex, introduced in [19].

Definition 2.1 *A reflex vertex v of P is a reflex extreme if there exists a vertical or horizontal line of support of v that is interior to P.*

For type 1, a reflex extreme vertex of P touches a side of the MAAPR and stops growth in one direction; we call this a *reflex contact*. Each reflex contact can remove one degree of freedom. Two reflex contacts with adjacent sides of the MAAPR fix a corner of the MAAPR. For type 2, a corner of the MAAPR touches an edge of P forming an *edge contact*.

2.2 Determining Sets of Contacts

Definition 2.2 *A set of contacts C is a determining set of contacts if the MAAPR R satisfying C has finite area and if the MAAPR R' satisfying any proper subset $C' \subset C$ has greater area.*

By way of example, we observe that a set of four reflex contacts, one on each side of the MAAPR, is a determining set.

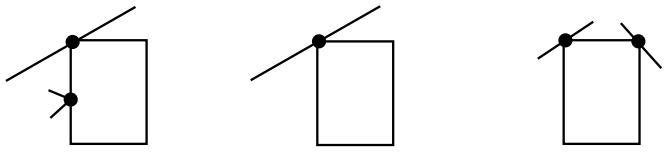


Figure 3: Edge Contact Types for a Determining Set

Within a determining set, we distinguish between two different subtypes of type 2 contacts. A contact is *fixed* if the set of constraints associated with the contact uniquely determines the point

of contact with the rectangle. Otherwise, it is a *sliding contact.*

A *fixed contact* can arise when there is no freedom to slide along an edge because a reflex contact fixes a coordinate (see Figure 3(a)). An edge contact with an adjacent side that has either a reflex or fixed contact must also be a fixed contact.

Two contacts are *dependent* if the position of one determines the position of the other; otherwise they are *independent.* An independent sliding contact requires that the two adjacent sides of the MAAPR do not have any contact with P (see Figure 3(b)). A sliding contact with an adjacent sliding contact is dependent, because the two contacts must share a coordinate (see Figure 3(c)).

2.3 Maximization Problems

Here we examine maximization problems associated with certain determining sets of contacts. Finding the MAAPR associated with a determining set of contacts requires solving a maximization problem if the set contains a sliding contact. For a given set of contacts, the number of degrees of freedom is the number of undetermined parameters of the rectangle. Degrees of freedom within a determining set can only arise from sliding contacts because any other degree of freedom would result in a rectangle of infinite area and therefore the set of contacts would not be determining. It follows that if a determining set consists of only reflex or fixed edge contacts, no maximization is required. For each independent sliding contact in the set, we parameterize the associated edge. The maximization problems can then be classified based on the number of parameters.

2.3.1 One Parameter Problems

The set of 1-parameter maximization problems can be further subdivided according to the number of dependent sliding contacts.

The Basic One Parameter Problem:

The simplest 1-parameter problem involves no dependent sliding contacts; just a single independent one. This is the basic 1-parameter problem, and it arises when one corner of the MAAPR has a

sliding contact and the opposite corner is fixed either by a pair of adjacent reflex contacts or by a reflex and a fixed contact. The basic 1-parameter problem can be solved by parameterizing the edge associated with the sliding contact and maximizing a quadratic in one variable. This can be solved in O(1) time.

An alternate constant-time solution to the 1-parameter problem is based on the following lemma, which demonstrates that the slope of the MAAPR diagonal depends only on the slope of the polygon edge. We assume here that the edge is neither vertical nor horizontal.

Lemma 2.1 (Slope Lemma) *Given a point p and a line L with slope s, the MAAPR with one corner at p and opposite corner at point q on L has diagonal $\overline{pq}$, where the slope of $\overline{pq} = -s$ (see Figure 4(a)).*

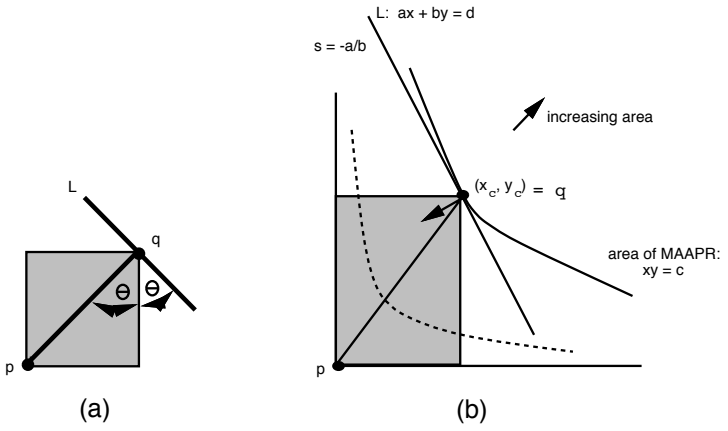


Figure 4: Slope Lemma

Proof: Assume p is at the origin, and that the line L is given by $ax + by = d$ (see Figure 4(b)). W.l.o.g. assume L intersects the $+x$ and $+y$ axes. (Note: we ignore the degenerate cases of horizontal and vertical lines, for which the MAAPR is undefined.) The family of hyperbolas given by $xy = r$ represents curves corresponding to rectangles of constant area. Moving away from the origin into the first quadrant, the hyperbola branch which is tangent to L provides the area c of the MAAPR associated with L. Let (x_c, y_c) be the coordinates of the point on L tangent to the hyperbola $xy = c$.

Now, the tangent to $xy = c$ at (x_c, y_c) is collinear with L, so the normal to $xy = c$ at (x_c, y_c) is collinear with the normal to L at (x_c, y_c). The normal to $xy = c$ is $(F_x, F_y) = (y, x)$; evaluated at (x_c, y_c) it is (y_c, x_c). The normal to L at (x_c, y_c) is (a, b). Therefore,

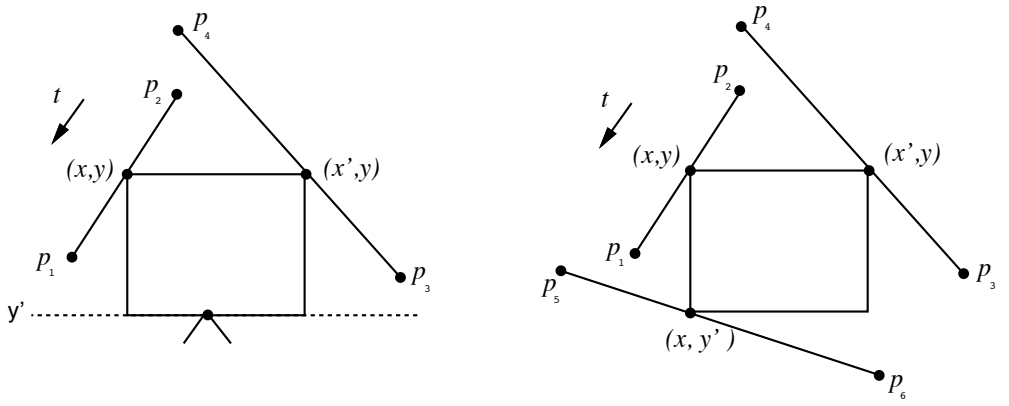
$$\text{slope}(\overline{pq}) = \frac{y_c}{x_c} = \frac{a}{b} = -\text{slope}(L).$$

■

Note that, as a consequence of this lemma, if p moves downward, q moves downward along the edge.

Two Dependent Sliding Contacts:

Consider a pair of dependent sliding contacts which share a coordinate (w.l.o.g. y), and have an opposite reflex contact which determines one coordinate of a side, (w.l.o.g. y') (see Figure 5(a)).



(a) Pair of Constrained Sliding Contacts (b) Triple of Constrained Sliding Contacts

Figure 5: 1-parameter problems with two and three dependent sliding contacts

To find the MAAPR, we parameterize the edge $\overline{p_2 p_1}$ by t, yielding the following quadratic in t to maximize:

$$\begin{aligned} \mathcal{F}(t) &= (x' - x)(y - y') \\ &= (A + Bt)(C + Dt) \end{aligned} \tag{1}$$

where:

$$\begin{array}{lllllll} A & = & \frac{y_2-b}{m} - x_2 & B & = & \frac{y_1-y_2}{m} + (x_2 - x_1) \\ C & = & y_2 - y' & D & = & y_1 - y_2 \\ m & = & \frac{(y_4-y_3)}{(x_4-x_3)} & b & = & y_4 - mx_4 \end{array}$$

Note that x' is obtained by solving the line equation associated with edge $\overline{p_4p_3}$.

Three Dependent Sliding Contacts:

For the case of three dependent sliding contacts, (see Figure 5b), we again parameterize edge $\overline{p_2p_1}$ and express x' in terms of y for edge $\overline{p_4p_3}$. Similarly, we express y' in terms of x for edge $\overline{p_5p_6}$. We again obtain a quadratic in t to maximize. The difference between this and the previous case is in the values of C and D:

$$\begin{array}{lllllll} C & = & y_2 - m'x_2 - b' & D & = & y_1 - y_2 + m'(x_2 - x_1) \\ m' & = & \frac{(y_6-y_5)}{(x_6-x_5)} & b' & = & y_5 - m'x_5 \end{array}$$

2.3.2 The Two Parameter Problem

There is only one type of 2-parameter problem. It has two opposite independent sliding contacts. The following lemma allows us to reduce a 2-parameter problem to a set of 1-parameter problems.

Lemma 2.2 *Let e_1 and e_2 be nonintersecting line segments. Consider the set E of axis-parallel rectangles which have opposite corners on e_1 and e_2. There is a maximum area rectangle in E with at least one corner at an endpoint of e_1 or e_2.*

Proof: One way to prove the claim is to parameterize each edge to obtain a quadratic in two variables. It is easily shown that the resulting surface is a saddle surface, so that the maximum is achieved along the boundary of the unit patch, implying that the MAAPR has a corner at a vertex of one of the edges.

A geometric argument is more intuitive, however. Let c_1 on e_1 and c_2 on e_2 be opposite corners of a rectangle, and let r be the diagonal of connecting c_1 and c_2. Now replace e_1 and e_2 by the lines

l_1 and l_2 containing them. Consider the set S of all line segments that connect l_1 and l_2 and are parallel to r. The length of these line segments as a function of their distance from r is monotonic. If l_1 and l_2 are parallel, the length is constant. Otherwise the length increases moving away from r in one direction, and decreases in the opposite direction. If neither c_1 nor c_2 is at a vertex of e_1 or e_2, move away from r in the direction of increasing length (either direction if l_1 is parallel to l_2) until either c_1 or c_2 is at a vertex of e_1 or e_2.

■

Having established that the MAAPR has a corner at a vertex in this case, we can find the MAAPR by considering, in turn, each of the four endpoints of e_1 and e_2, solving the associated 1-parameter problems, and then comparing the four resulting 1-parameter MAAPR areas.

2.4 Characterization Theorem

To characterize the MAAPR, we examine the possible determining sets of contacts. By enumerating the reflex contacts between the MAAPR and P, we derive the set of five cases shown in Figure 6.

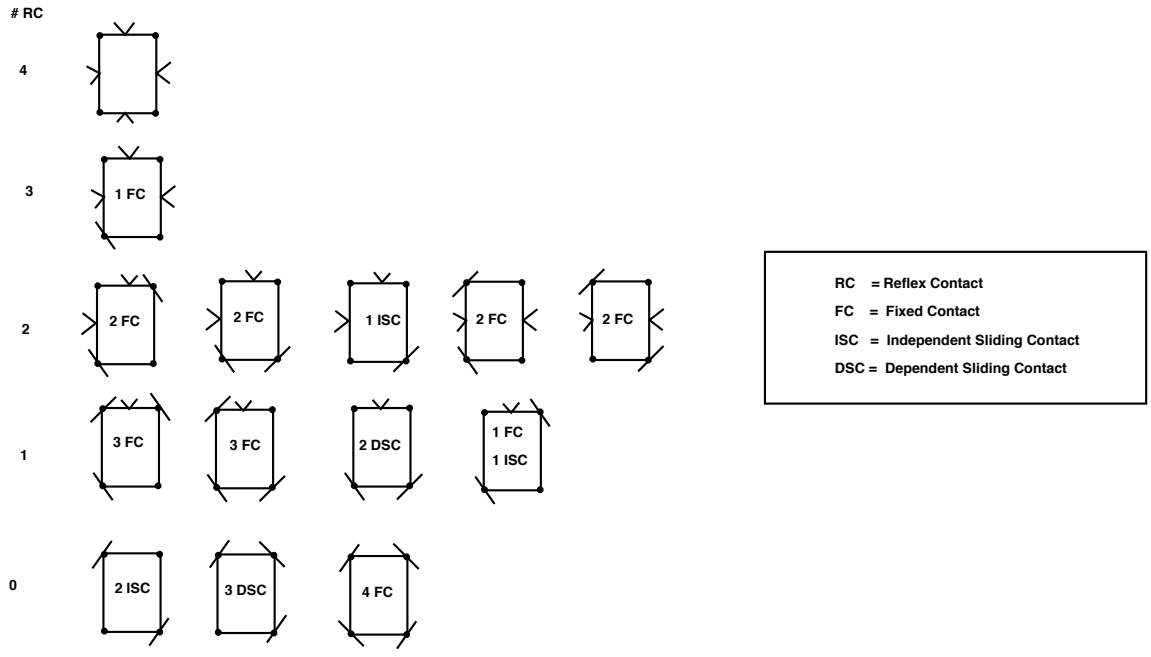


Figure 6: Determining Sets of Contacts for the MAAPR

Theorem 2.3 *The determining set of the MAAPR of a general polygon P conforms (up to symmetry) to one of the five cases in Figure 6.*

Proof: A determining set has, by definition, at most one reflex contact with each side of the MAAPR. For each of the possible numbers of reflex contacts, we show that the determining set of a MAAPR of that type conforms (up to symmetry) to one of the configurations shown in Figure 6. In each case, we must eliminate degrees of freedom beyond those of sliding contacts. We observe that a determining set cannot contain both two adjacent reflex contacts and the fixed contact between them; this would be redundant.

Case 4: In this case there is one reflex contact with each side of the MAAPR. Since each reflex contact removes one degree of freedom from the rectangle, this set of contacts is sufficient to determine the MAAPR. Removing any one reflex contact allows the rectangle to grow, so all four contacts are necessary.

Case 3: Three reflex contacts are not sufficient to determine the MAAPR, since the fourth side can grow. We must add an edge contact which is fixed because it is adjacent to a reflex contact. There is only one choice of position for this contact, up to symmetry. It fixes the remaining side of the rectangle.

Case 2: Two reflex contacts can be either adjacent or opposite. In both cases, edge contacts are needed to determine the MAAPR. Since two degrees of freedom remain, the determining set contains at most two edge contacts.

In the adjacent case, we examine the possibility of edge contacts at the corners of the rectangle, excluding the corner between the two reflex contacts. There are two ways that two edge contacts can appear. In both cases both contacts must be fixed, so they fix the remaining sides of the rectangle. If only one edge contact appears, it must be a sliding contact; otherwise a degree of freedom remains. The sliding contact is opposite to the corner fixed by the reflex contacts. This contact is sufficient because it represents a basic 1-parameter problem.

In the opposite case, any edge contact is fixed; hence there must be two of them. There are two possible ways they can be configured. In both cases, all four sides of the rectangle are fixed.

Case 1: One reflex contact is not sufficient to determine the MAAPR. We can choose to place edge contacts at any three of the four corners of the rectangle (four would be redundant). If there are three edge contacts, they are all fixed and they completely determine the MAAPR. There are two ways to configure three such contacts. If there are two edge contacts, they can be adjacent or opposite. If they are adjacent, they must both be opposite the reflex contact; otherwise a degree of freedom remains. These adjacent contacts are dependent sliding contacts, and they determine the MAAPR because they represent a 1-parameter problem. If they are opposite, one is fixed, and the other is a sliding contact; this is also a 1-parameter problem. One edge contact is not sufficient, together with the one reflex contact, to determine the MAAPR.

Case 0: In this case there are no reflex contacts. One sliding contact is not sufficient to determine the MAAPR. If there are two sliding contacts, they must be opposite if they are to determine the MAAPR. This is a 2-parameter problem. If there are three edge contacts, they are all dependent sliding contacts. They are sufficient to determine the MAAPR because they represent a 1-parameter problem. If there are four edge contacts, they yield four equations in four unknowns, for which the MAAPR is completely determined. This therefore forms a set of fixed contacts.

■

Corollary 2.4 *Given a determining set C for a MAAPR, $2 \leq |C| \leq 4$.*

2.5 A Naive Algorithm

Based on the above characterization, we can find the MAAPR in a polygon by finding the MAAPR under the constraints of each of the five cases and selecting the largest one.

Theorem 2.5 *For each determining set of contacts in Figure 6, the MAAPR can be found in constant time.*

Proof: A reflex contact yields, in constant time, one of the four parameters of the rectangle, as does a fixed contact. In all cases we can process these contacts first. The remaining situations

are all either 1-parameter problems or 2-parameter problems, which can be solved in constant time because they only require maximizing a constant number of quadratic forms. ∎

A naive MAAPR algorithm can use Theorem 2.5 and supply it, in each case, all possible determining sets for P. These can be identified using an algorithm with four nested loops; one for each side of the MAAPR. With $\Theta(n)$ preprocessing time to build horizontal and vertical visibility maps [7], we can determine in $O(1)$ time, given a set of edges for a case, the pieces of those edges which are candidates for determining the MAAPR. We conclude:

Theorem 2.6 *The MAAPR of an n-vertex general polygon can be found in $O(n^4)$ time.*

In the remainder of the paper we show how to use the MAAPR characterization combined with fast matrix searching to develop a more sophisticated approach to this problem.

3 A General Framework for the 2-Contact Case

As the previous section detailed, there are many different cases to consider when determining the MAAPR in a general polygon. For rectilinear polygons, the two-contact cases dominate the running time. This is also true for non-rectilinear polygons; here the case of two independent sliding contacts determines the running time. Finding largest empty rectangles with three or more contacts can be done faster and does not require substantially new ideas. We defer these cases to Section 5. We also know from the previous section that we only need to consider two-contact solutions in which one corner of the rectangle is at a vertex and the opposite corner is touching an edge of the polygon. In particular, Lemma 2.2 implies that the two sliding contact case (edge-edge) can be reduced to solving four of these single sliding contact problems (corner-edge or side-vertex).

Specifically, for three types of polygons we consider the following problem. Given two disjoint chains from a polygon P, find an empty[5] rectangle inside P at least as large as the largest area empty rectangle which has its lower-left corner at a vertex in the first chain and its upper-right corner

[5]A rectangle is empty if its interior does not contain any point of P.

on an edge of the second chain. For obvious reasons, we call the first chain the *vertex chain* and the second chain the *edge chain*, and we call the rectangle a *two-chain vertex-edge* MAAPR. The polygons we consider and the running times we obtain are as follows:

i. XY-monotone polygon made of two monotonically decreasing chains. Vertex chain: lower chain. Edge chain: upper chain. Running time: $\Theta(n)$.

ii. Orthogonally convex polygon. Vertex chain: monotonically decreasing "lower-left" chain of polygon. Edge chain: monotonically decreasing "upper-right" chain of polygon. Running time: $O(n\alpha(n))$.

iii. Horizontally convex polygon with left and right chains (extending from highest to lowest point of P) on opposite sides of some vertical line V. We call this a *vertically separated, horizontally convex* polygon. Vertex chain: chain to the left of V. Edge chain: chain to the right of V. Running time: $O(n \log n)$.

Note that unlike types i and iii, the vertex and edge chains in the type ii polygon do not constitute the entire polygon.

We first give a general "meta-theory" for solving a class of problems which includes these three types of polygons. Then we specifically address the details for each of the above polygons.

3.1 Meta-Theory

We wish to solve the two-chain vertex-edge MAAPR problem, as described above, for a polygon P. Our first task is to transform P into a polygon P' (the chains are altered too) such that interior(P') $\subseteq$ interior(P) and properties I-III (defined below) are satisfied. For the set of vertices of P' we then consider the problem of finding the largest-area empty corner rectangle (LAECR). The LAECR for a set of points is well-known in the literature as a subproblem of the largest empty rectangle problem [8, 3]. The LAECR problem is stated as follows: given two sets of points L and R, find the MAAPR containing no point of the sets which has lower left corner in L and upper right corner in R.

Finding the LAECR for a set of points requires an *area oracle*, which, given a point l in L and a point r in R, computes, in constant time, the area of a rectangle associated with l and r.

We show in our meta-theorem (Theorem 3.1) that if P' satisfies properties I-III, we can construct such a MAAPR area oracle (see Definition 3.1) and that running a LAECR algorithm on P' with this oracle produces (in a sense that is made exact in the statement of the theorem) a solution for the two-chain vertex-edge MAAPR of P.

3.1.1 Meta-Theorem

We now define properties I-III, define a MAAPR area oracle, and prove the meta-theorem.

Property I: Each two-chain vertex-edge rectangle for P is a two-chain vertex-edge rectangle for P'. (We consider a vertex-vertex rectangle also to be a vertex-edge rectangle because if the upper corner is touching a vertex, it is also touching two edges.)

Property II: For every vertex u in the vertex chain of P' and every edge e in the edge chain of P': if any point q in the interior of e is rectangularly visible[6] from u inside P', then the entire edge e is rectangularly visible from u.

Property III: If vertex u in the vertex chain of P' and a point q on the edge chain of P' (to the upper-right of u) are rectangularly visible *with respect to the set of vertices of* P', then u and q are rectangularly visible with respect to P'.

Definition 3.1 (MAAPR Area Oracle) *Add a* **special** *vertex at the midpoint of every edge in the edge chain of* P'. *Define a* **MAAPR area oracle** *which computes the area of rect*(uv) *for a vertex* u *in the vertex chain and a vertex* v *in the edge chain as follows*[7]. *If* v *is an ordinary vertex, the oracle returns the ordinary area. If* v *is a special vertex, the oracle returns the area of the MAAPR for vertex* u *and the edge whose upper endpoint is* v.

The MAAPR area oracle clearly runs in constant time (see Lemma 2.1).

[6] Two points p and q have *rectangular visibility* [17] if the interior of the axis-parallel rectangle with diagonal $\overline{pq}$ does not intersect the polygon.

[7] rect(uv) is the rectangle with lower left corner at u and upper right corner at v.

Theorem 3.1 (Meta-Theorem) *Assume the notion of an area is the one defined by the MAAPR area oracle. Then, computing the LAECR of the vertices of P', using this area oracle yields the area of a rectangle which is at least as large as the largest empty two-chain vertex-edge rectangle inside P.*

Proof: Let S be the largest empty vertex-vertex rectangle inside P', under the notion of the MAAPR area oracle, and T be the largest empty two-chain vertex-edge rectangle, under the notion of the ordinary area oracle. We show that $Area(S) = Area(T)$ (under the notion of the ordinary area oracle). From this the theorem follows, because we know from property I that the area of T is at least as large as the largest two-chain vertex-edge rectangle inside P, and property III implies that empty with respect to the vertices of P' means empty with respect to the polygon P'.

$Area(S) \geq Area(T)$: Let uq be the diagonal of T where q lies on edge e of the edge chain. Let v be the special vertex of e. By property II, u can see v, so the algorithm must have returned an area at least as big as the MAAPR oracle area for rect(uv). But that equals the ordinary area of rect(uq).

$Area(T) \geq Area(S)$: We must show that there is a two-chain vertex-edge rectangle rect(uq) with the area returned by the algorithm. Let uv be the diagonal of the rectangle returned by the algorithm. If v is ordinary, then rect(uv) is the one we want. If v is special, since u can see v, u can see all of edge e containing v, and thus u can see q where uq is the diagonal of the MAAPR for u and e. ■

3.1.2 Computing LAECR

To compute the MAAPR efficiently, we want to simulate efficient known LAECR algorithms and use the "meta-theorem" (Theorem 3.1). The reason this can be done, is that the LAECR algorithms we use only require that the "area matrix" M, whose entry m_{ij} contains the area computed by area oracle given vertex u_i and upper-right vertex v_j, satisfies a certain *monotonicity* property. Of course, some entries in the area matrix are invalid because u_i and v_j are not rectangularly visible. However, the only property which the algorithms really depend on is the *monotonicity* property for legal[8] 2×2 minors of the matrix.

[8] A legal 2×2 minor contains only entries corresponding to empty rectangles.

Next, we define this property and show that M defined by the MAAPR area oracle satisfies it.

Definition 3.2 ([2]) *M is* **totally monotone**[9] *if, for every $i < i'$ and $j < j'$ corresponding to a legal 2×2 minor, $m_{i'j'} > m_{i'j}$ implies $m_{ij'} > m_{ij}$.*

We state the following lemma:

Lemma 3.2 *If an increasing index in M corresponds to decreasing y-coordinates of the associated vertices, then M defined by the MAAPR area oracle is totally monotone.*

Proof: It suffices to assume that the two edge chain vertices are special vertices, since we can consider an ordinary vertex to represent an edge of zero length with a special vertex equal to the ordinary vertex. The first step follows the argument in [3]. By assumption m_{ij}, $m_{ij'}$, $m_{i'j}$, and $m_{i'j'}$ are all empty rectangles. Since m_{ij} is empty and $u_{i'}$ is below u_i, $u_{i'}$ must be to the right of u_i. Since m_{ij}, $m_{ij'}$ and $m_{i'j}$ are empty and $v_{j'}$ is below v_j, $v_{j'}$ must be to the right of v_j.

Let p_1, p_2, p_3, and p_4 be the MAAPR vertices for rect$(u_i v_j)$, rect$(u_{i'} v_j)$, rect$(u_i v_{j'})$ and rect$(u_{i'} v_{j'})$, respectively. Let A, B, C, D, E, and F be the areas of rect(u_i, p_1), rect(u_i, p_3), rect$(u_{i'}, p_2)$, rect$(u_{i'}, p_4)$, rect(u_i, p_2), and rect$(u_{i'}, p_3)$. respectively. To show monotonicity, it suffices to show that: $B > A \Rightarrow D > C$. To show this we need the intermediate result: $B > E \Rightarrow F > C$.

$$\begin{aligned} B &> E \\ (p_{3x} - u_{ix})(p_{3y} - u_{iy}) &> (p_{2x} - u_{ix})(p_{2y} - u_{iy}) \\ p_{3x}p_{3y} + u_{ix}(p_{2y} - p_{3y}) &> p_{2x}p_{2y} + u_{iy}(p_{3x} - p_{2x}) \end{aligned}$$

But, since we know $u_{i'x} \geq u_{ix}$, $u_{iy} \geq u_{i'y}$, $p_{2y} \geq p_{3y}$, and $p_{3x} \geq p_{2x}$, we have:

$$p_{3x}p_{3y} + u_{i'x}(p_{2y} - p_{3y}) > p_{2x}p_{2y} + u_{i'y}(p_{3x} - p_{2x})$$

[9] In the literature, the term *totally monotone* refers to a matrix which has the total monotonicity property and no illegal entries. Matrices which have the property but have some illegal entries are referred to as *monotone* (e.g. monotone-single-staircase, monotone-double-staircase). This is confusing, since monotonicity is also presented in the literature as a weaker condition than total monotonicity: a matrix for which the column of the row maximum moves to the right as i increases is monotone, and it is totally monotone only if all 2×2 minors also possess this property. Our terminology removes this confusion.

$$\begin{aligned}(p_{3x} - u_{i'x})(p_{3y} - u_{i'y}) &> (p_{2x} - u_{i'x})(p_{2y} - u_{i'y}) \\ F &> C\end{aligned}$$

The definition of a MAAPR implies that: $A \geq E$ and $D \geq F$. $B > A$ and $A \geq E \Rightarrow B > E \Rightarrow F > C$. $D \geq F$ and $F > C \Rightarrow D > C$. Therefore $B > A \Rightarrow D > C$. ■

Corollary 3.3 *If the vertex and edge chains of P' are y-monotone, then M defined by the MAAPR area oracle is totally monotone.*

Proof: P' satisfies the condition of Lemma 3.2. ■

3.1.3 MAAPR Running Time

Based on Theorem 3.1 and Corollary 3.3, we make the following claim, which is used in Section 3.2, Section 3.3, and Section 3.4 to establish running times for polygons of types i, ii, and iii.

Claim 3.4 *For an n-vertex polygon P, if we can produce an $O(n)$ vertex polygon P' satisfying properties I-III in $O(n)$ time, and if the LAECR algorithm depends only on the total monotonicity of the matrix associated with the MAAPR oracle, then, if the vertex and edge chains of P' are y-monotone, the two-chain vertex-edge MAAPR of P can be found in the same asymptotic running time as the LAECR algorithm.*

In general, we produce P' for types i, ii, and iii by projecting vertices of P and replacing some edges of P by inner rectilinear approximations. For this, we assume $O(n)$ preprocessing time to construct horizontal and vertical visibility maps for P. Projection and rectilinearization can be done in $O(n)$ time and add only a linear number of vertices to P. For types i, ii, and iii, the vertex and edge chains are y-monotone, and the LAECR algorithms we use depend only on the total monotonicity of the area matrix. To obtain the desired running time in each case for finding the two-chain, vertex-edge MAAPR, it therefore suffices to construct P', show it satisfies properties I-III, and establish the running time of the appropriate LAECR algorithm.

Implementation Note: We can avoid the use of special vertices if we accept a more complicated oracle. For entry m_{ij}, let e be the edge whose upper endpoint is v_j. If all of e is rectangularly visible to u_i, this oracle returns the area of the MAAPR whose lower-left corner is at u_i and upper-right corner is on e. Otherwise, it returns the area of $rect(u_i v_j)$.

3.2 Two XY-Monotone Chains

The first polygon we consider is xy-monotone and has two decreasing xy-monotone chains. The lower chain is the vertex chain and the upper chain is the edge chain.

3.2.1 Construction of P'

Given such a polygon P, we transform it into P' as follows. From each vertex in the vertex chain of P, project a horizontal ray rightward and a vertical ray upward. Add vertices to the edge chain where these two rays hit it. Clearly this adds at most $2n$ new vertices.

Proof of Property I: Property I is easily verified for P and P' since we have merely subdivided the edges of the edge chain.

Proof of Property II: Property II is also easily proven. Let u be a vertex on the vertex chain and let q and q' be two points on the same edge e on the edge chain. We need to show that q is visible if and only if q' is. Suppose q is visible and q' is not. Imagine moving q towards q'. At some point rect(uv) must degenerate into either a horizontal or vertical line segment. By the construction of P', there must be a vertex at this point of e. Therefore q and q' do not lie on the same edge, producing a contradiction.

Proof of Property III: Property III follows from the xy-monotonicity of the chains. If u is any vertex on the lower chain and v is any vertex on the upper chain such that u is below and to the left of v, then the rectangle they form is necessarily empty of both vertices and edges.

3.2.2 LAECR Algorithm

Finally, we describe an algorithm for finding the LAECR with a vertex chain vertex u on the lower chain and an edge chain vertex v on the upper chain. It is well-known that if all points u_i lie below-left of all points v_j, then the area matrix $m_{ij} = area(rect(u_i v_j))$ is not only *totally monotone*, but, in addition, *every* 2×2 minor is legal [2]. If any u_i lies either above-left or below-right of any v_j($u_i v_j$ has negative slope), then we give the area the same magnitude but negative sign.

Note that some entries of the resulting area matrix correspond to illegal rectangles, but these entries are all negative. An algorithm for finding the maximum entry will return a positive entry, which will correspond to a legal rectangle. Because the chains are xy-monotone, the matrix is still totally monotone even though some entries are illegal and negative.

We have thus reduced the problem to that of finding the maximum entry in an $n_1 \times n_2$ totally monotone matrix where n_1 and n_2 and the lengths of the two chains. We discuss in Section 4 how this can be done in $O(n_1 + n_2)$ time, yielding a $\Theta(n)$ time algorithm for finding the two-chain vertex-edge MAAPR for an xy-monotone polgyon.

Implementation Note: The set of illegal entries in the matrix is bounded by falling staircases. Aggarwal and Klawe [1] show that such a monotone matrix can be *completed* so that its maximum can be found in linear time. Using negative entries is our practical alternative to using completion here. In fact, for this particular problem one can drop any notion of visibility and simply use the vertices of P. The projection step is unnecessary. The use of signs to represent illegality makes this work naturally.

3.3 Two Chains of an Orthogonally Convex Polygon

An orthogonally convex polygon P is bounded by four xy-monotone chains. Let the lower-left chain be the vertex chain and the upper-right chain be the edge chain.

3.3.1 Construction of P'

To create the polygon P', for each vertex u in the lower-left (vertex) chain, do the following: project rightwards and upwards, creating two new vertices. For each vertex w on the upper-left chain (including the newly created ones) do the following: if the rightward projection of w hits the upper-right (edge) chain, project w rightwards and create a new vertex on the edge chain. Similarly, for each vertex w on the lower-right chain, project it upwards if it lies below the edge chain.

Replace every edge ab (a to the left of b) of the upper-left chain by a horizontal edge ac followed by a vertical edge cb. (This must be done *after* the projection step.) In other words, replace the upper-left chain by an inner rectilinear approximation. Similarly, replace the lower-right chain by an inner rectilinear approximation. The final result is P'.

Proof of Property I: We do not change the shape of the vertex and edge chains, although we do add a linear number of new vertices to the edge chain. Therefore a two-chain vertex-edge rectangle of P is also a two-chain vertex-edge rectangle of P'. We only have to verify that if it is empty in P, then it is empty in P'. Suppose vertex-edge rectangle $rect(uq)$ is empty in P, but not empty in P'. Then rect(uq) must contain a vertex c of the rectilinearized upper-left (w.l.o.g.) boundary of P', but it does not intersect edge ab of P, where acb is the inner rectilinear approximation of ab. Therefore $a_x < u_x < c_x$. But this cannot be because u was projected upwards, so there should be a vertex between a and b with x-coordinate u_x.

Proof of Property II: Let u be a vertex of the vertex chain and let q and q' be two vertices on the same edge e of the edge chain of P'. We know from the proof of property II for xy-montone polygons that uq has positive slope if and only if uq' does. We need to show that if rect(uq) is empty, then rect(uq') is also. Because the construction of P' is symmetric with respect to x and y, we consider w.l.o.g. q below q'. Assume rect(uq) is empty but rect(uq') contains a vertex c of the upper-left chain. As we shift q towards q', the top edge of rect(uq) moves upwards and must hit c before q reaches q'. But $c_y = a_y$, where acb is the inner rectilinear approximation of ab, and a was projected rightwards onto the edge chain. Therefore q hits a projected vertex before it reaches q', contradicting the assumption that they were on the same edge.

Proof of Property III: Suppose rect(uv) contains no vertices of P' where u is a vertex of the

vertex chain lying lower-left from v, a vertex of the edge chain. Rectangle rect(uv) clearly does not intersect the monotonically decreasing vertex and edge chains. Since the upper-left and lower-right chains of P' are monotonic and rectilinear, it does not intersect them either.

3.3.2 LAECR Algorithm

If no entries were illegal, we would use the same algorithm as described for the xy-monotone chains in Section 3.2. However, we prove that the two additional xy-monotone chains in the current case introduce two sets of illegal entries into the area matrix. We show that the boundaries of these sets are rising staircases, so that the matrix is a double staircase matrix (see definition below). We again represent entries whose rectangle diagonals have negative slope by assigning them negative area.

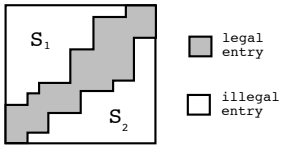


Figure 7: Totally Monotone Rising Double Staircase Matrix

Definition 3.3 ([3]) [10] *A matrix M is* **totally monotone double staircase** *if it is totally monotone and if there exist two sets of illegal entries S_1 and S_2 such that the boundary of each set forms an xy-monotone staircase inside M and S_1 and S_2 lie in diagonally opposite corners of M.*

Figure 7 shows a totally monotone *rising* double staircase matrix (S_1 and S_2 are in the upper left and lower right corners, respectively). Note that the two staircases may intersect, although they do not in the figure.

Before giving the LAECR algorithm, we prove that the area matrix for this case is a monotone rising double staircase matrix.

Lemma 3.5 *M is a totally monotone rising double staircase matrix.*

[10] Aggarwal and Suri actually use the term *monotone double staircase* matrix. See our earlier note on total monotonicity.

Proof: Let rect($u_i v_j$) be a rectangle that does not intersect the upper-left chain and let q be the upper-left corner of that rectangle. Every rectangle rect($u'_i v'_j$) for $i' \geq i$ and $j' \geq j$ lies in the lower right quadrant of q and therefore does not intersect the upper-left chain. Thus if any entry in the area matrix is legal with respect to the upper-left chain, then every entry to its lower-right in the matrix is also legal. Therefore the set of entries which are illegal with respect to the upper-left chain form the set S_1.

It is straightforward to show that the lower-right chain produces the set S_2. ■

The LAECR algorithm in this case must find the staircase boundaries of M in linear time and then find the maximum entry in M. In Section 4 we show how to find the maximum value in a totally monotone rising double staircase matrix in $O(n\alpha(n))$ time. We can construct the left staircase (w.l.o.g.) in linear time by traversing the upper-left chain as follows. For row i, we obtain u_i on the vertex chain and project up and to the right to obtain v_j on the edge chain. (The upward projection might hit the edge chain, directly yielding v_j. It is also possible that the rightward projection hits the lower-right chain; in this case the entire row is illegal.) Entry i, j is the first legal entry in row i with respect to the upper-left chain.

Thus, we obtain, by Claim 3.4, an $O(n\alpha(n))$ time algorithm for finding the two-chain, vertex-edge MAAPR for two chains of an orthogonally convex polygon.

3.4 Two Chains of a Vertically Separated, Horizontally Convex Polygon

Consider now a vertically separated, horizontally convex polygon. The left chain L is the vertex chain and the right chain R is the edge chain. Note that L and R are y-monotone.

3.4.1 Construction of P'

We produce the modified polygon P' using the following set of projections followed by rectilinearization. From each vertex in the vertex chain L, project a vertical ray upwards, adding vertices

where the rays hit L. For each vertex in L (including new ones) project a horizontal ray rightward. Add vertices to the edge chain R where these rays hit. For each vertex of R (including new ones) project up, adding new vertices. Now, replace each edge of modified L and R which has positive slope by its inner rectilinear approximation to produce P'. This process adds a linear number of new vertices.

Proof of Property I: Edges of P which have negative slope are not rectlinearized, so a two-chain vertex-edge rectangle of P is also a two-chain vertex-edge rectangle of P'. We need only show that if it is empty in P it is empty in P'. This argument is identical to that in the proof of Property I for the orthogonally convex case, and is omitted.

Proof of Property II: This is similar to the proof of Property II for the orthogonally convex case. However, since the construction of P' is not symmetric in x and y, we must also consider the case where q is above q'. Assume rect(uq) is empty but rect(uq') contains a vertex c of the lower-right chain. As we shift q towards q', the right edge of rect(uq) moves rightwards and must hit c before q reaches q'. But $c_x = b_x$, where acb is the inner rectilinear approximation of ab, and b was projected upward to the edge chain. Therefore q hits a projected vertex before it reaches q', contradicting the assumption that they were on the same edge.

Proof of Property III: The proof of Property I guarantees that rectangle rect(uv) does not intersect the inner rectilinear approximation of any positive slope subedge of P. It remains to show that no edge with negative slope can cut across rect(uv). A negative slope edge cannot cross the vertical line separating L and R, so it must cross either the upper-right or lower-left corner, contradicting the y-montonicity of one of the chains.

3.4.2 LAECR Algorithm

Aggarwal and Suri have an $O(n \log^2 n)$ algorithm for the Largest Empty Corner Rectangle (LAECR) problem[3, 4] for points. They use a divide-and-conquer approach which partitions the set of points into two subsets L and R about a vertical line V, and recursively finds the LAECR in L and R. The merge step requires finding the LAECR whose lower-left corner is in L and upper-right corner is in R. They solve this subproblem in $O(n \log n)$ time using a procedure we call LAECR_Merge

by *implicitly* forming an area matrix whose legal 2×2 minors are monotone and applying fast searching techniques to it. We pass to LAECR_Merge the points of the left chain, the points of the right chain, and implement it using our MAAPR area oracle.

By Claim 3.4, this yields an $O(n \log n)$ time algorithm for finding the two-chain, vertex-edge MAAPR in a vertically separated, horizontally convex polygon.

4 Solving the Matrix Problem

In this section we discuss efficient ways from the literature to compute the maximal elements in totally monotone matrices. These results form the basis of the algorithms presented in Section 5.

Naively, we can examine all $O(n^2)$ matrix entries to find the maximal element. However, Aggarwal et. al. showed that, if a matrix is totally monotone and has no illegal entries, the maximum can be found in linear time:

Lemma 4.1 ([2]) *If any entry of a totally monotone matrix of size $n \times m$ can be computed in constant time, then the row-maximum problem for this matrix can be solved in $\Theta(m)$ time if $m \geq n$ and $\Theta(m(1 + \log(n/m)))$ time if $n > m$.*

Note that this result assumes the existence of an oracle which takes as input (i, j) and returns the value m_{ij} in constant time.

Corollary 4.2 *If any entry of a totally monotone matrix M of size $n \times m$ can be computed in constant time, then the row-maximum problem for this matrix can be solved in $O(m + n)$ time.*

Proof: From Lemma 4.1, when $m \geq n$ the problem can be solved in $O(m)$ time. When $n > m$, the fact that $O(m(1 + \log(n/m))) \in O(n)$ immediately yields $O(n)$ time, which establishes the corollary. However, we present the following alternative proof for the $n > n$ case which is based on an algorithm that is easy to implement.

We can *complete* M using $-\infty$ by modifying the oracle. If $j \leq m$, it returns m_{ij}, but if $j > m$, the oracle returns $-\infty$. The oracle still operates in constant time.

The total monotonicity of the matrix is preserved because every 2×2 minor is monotone. To establish this, consider the minor given by: m_{ij}, $m_{i'j}$, $m_{ij'}$, $m_{i'j'}$, for $i' > i$ and $j' > j$. If the minor contains no $-\infty$ entries, it is monotone because M is totally monotone. If it has all $-\infty$ entries, then because $m_{ij} = m_{ij'}$ and $m_{i'j} = m_{i'j'}$ we cannot have the forbidden condition $m_{i'j} \geq m_{i'j'}$ when $m_{ij'} > m_{ij}$. In the remaining case the forbidden condition cannot occur because $m_{ij} > m_{ij'}$ and $m_{i'j} > m_{i'j'}$. The completed matrix has dimension $n \times n$, so we can find its maximum in $\Theta(n)$ time by Lemma 4.1. Combining the two cases yields $O(n + m)$. ■

This result is applied in Section 5 to the problem in Section 3 concerning two xy-monotone chains, with which we associated totally monotone matrices.

For the case of two chains of an orthogonally convex polygon, with which we associated totally monotone rising double staircase matrices, we also seek a subquadratic time search algorithm. Efficiently searching a totally monotone rising double staircase matrix involves transforming it into a set of rising *single* staircase matrices, defined below and illustrated in Figure 8. In the figure, *upper* and *lower* refer to the position of the set of illegal entries[11].

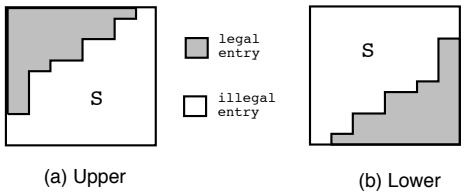


Figure 8: Totally Monotone Rising Single Staircase Matrices

Definition 4.1 ([3]) *A matrix M is* **totally monotone single staircase** *if there exists one special set of entries S such that any 2×2 minor that does not contain entries from S is totally monotone; the boundary of S forms an xy-monotone staircase inside M.*

Lemma 4.3 ([12]) *If any entry of a totally monotone single staircase matrix of size $n \times m$ can be computed in constant time, then the row-maximum problem for this matrix can be solved in $O(n\alpha(m) + m)$ time.*

[11] Aggarwal and Klawe [1] distinguish between rising (which we call upper) and reverse rising (lower) staircase matrices.

Implementation Note: Aggarwal and Suri note that the constants are large here. They recommend in practice a theoretically slower algorithm. One such algorithm is the $O((m+n)\log\log n)$ algorithm of Aggarwal and Klawe [1].

Lemma 4.4 ([4]) *Given a totally monotone rising double staircase matrix M for which the staircase boundaries are known, the row-maximum problem for M can be solved in $O(n\alpha(m)+m)$ time.*

The proof of this is based on the following claim and Lemma 4.3.

Claim 4.5 ([3]) *An $n \times m$ totally monotone rising double staircase matrix can be partitioned in $O(n+m)$ time using an xy-monotone chain into a set of at most $2k$ single staircase matrices, each of dimension $n_i \times m_i$, such that $\sum_{i=0}^{2k-1}(n_i) \leq 2n$ and $\sum_{i=0}^{2k-1}(m_i) \leq 2m$.*

Aggarwal and Klawe provide an alternate partitioning of a totally monotone rising double staircase matrix[1]. Their partitioning holds for the more general *totally monotone partial matrix*, which contains two sets of illegal entries whose boundaries are unimodal sequences. Klawe [11] extends the definition of a totally monotone partial matrix to any matrix whose set of legal entries is orthogonally convex. Without giving the details, she notes that the algorithm leading to Lemma 4.3 can be trivially extended to handle this type of matrix. We observe that this type can be accomodated via partitioning, without extending the algorithm, as follows:

Lemma 4.6 *An $n \times m$ totally monotone partial matrix M whose staircase boundaries are known can be partitioned in $O(n+m)$ time into a set of five matrices. One is a totally monotone double staircase matrix and the other four are totally monotone single staircase matrices.*

Proof: Let t, b, l and r be the top, bottom, leftmost and rightmost legal entries of M. Let p be the downward projection of t, and q be the upward projection of b. We can find p and q in $O(n+m)$ time. The matrix determined by $tqbp$ is a totally monotone double staircase matrix M'. M' partitions M, leaving triangular submatrices on the left and right. If q is on tr, then the triangular submatrix qrb is partitioned into two totally monotone single staircase matrices by the horizontal line through r and the triangular submatrix tlp is partitioned into two totally monotone

single staircase matrices by the horizontal line through l. If q is on tl, we partition qlb and trp into totally monotone single staircase matrices. Partitioning the triangular regions takes $O(n+m)$ time. ■

With this partitioning, one can find the maximum in a totally monotone partial matrix in $O(n\alpha(m)+m)$ time by simply using Lemma 4.3 and Lemma 4.4, which apply the algorithm of Lemma 4.3 *without* modifications.

5 Finding the MAAPR

In this section we use the results of the previous sections to derive efficient algorithms for computing the MAAPR in different classes of n-vertex polygons. We obtain the following running times (see also Figure 2):

- XY-monotone polygon: $\Theta(n)$.
- Orthogonally convex polygon: $O(n\alpha(n))$.
- Horizontally convex polygon (or vertically convex): $O(n\alpha(n)\log n)$.
- Vertically separated, horizontally convex polygon (or horizontally separated, vertically convex): $O(n\log n)$.
- General polygon with holes: $O(n\log^2 n)$.

For each type we use the characterization results of Section 2 to identify the relevant determining sets of contacts. For the 2-contact case, we use the results of Section 3 to reduce the problem to a LAECR problem and then obtain computational bounds for the associated matrix problem from Section 4. Finally, we show how to deal with cases of three or more contacts.

5.1 XY-Monotone Polygon

Theorem 5.1 *The MAAPR in an n-vertex xy-monotone polygon can be found in $\Theta(n)$ time.*

Proof: The MAAPR in an xy-monotone polygon must be a 2-contact MAAPR. From the results of Section 3.2 and Corollary 4.2 we can find the 2-contact MAAPR in $\Theta(n)$ time. ■

5.2 Orthogonally Convex Polygon

The boundary of an orthogonally convex polygon can be partitioned, in linear time, into four xy-monotone polygonal chains. We establish the following result for this type of polygon:

Theorem 5.2 *The MAAPR in an n-vertex orthogonally convex polygon can be found in $O(n\alpha(n))$ time.*

Proof: An orthogonally convex polygon has no reflex extreme vertices, so there are only three possible configurations for MAAPR determining sets of contacts (see Figure 9).

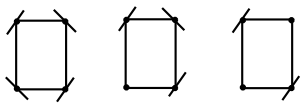


Figure 9: Determining Sets of Contacts for an Orthogonally Convex Polygon

We show that the 3 and 4-contact cases can be solved in linear time[12], and the 2-contact case can be solved in $O(n\alpha(n))$ time.

Lemma 5.3 *The 3-contact and 4-contact MAAPRs of an n-vertex orthogonally convex polygon can be found in $\Theta(n)$ time.*

Proof: The 3 and 4-contact MAAPRs have either both bottom corners on the bottom chains of P or both top corners on the top chains. A bottom-up sweep solves the bottom corners case, where events are vertices of the bottom chains. Each event determines a y-span along two edges e_1 and e_2. Let the top chain pieces corresponding to e_1 and e_2 be c_1 and c_2, respectively (see Figure 10). The natural y-ordering of vertices along c_1 and c_2 yields a set of secondary events on the upper chains

[12]McKenna et.al.[13] obtain the same time bound for the rectilinear case.

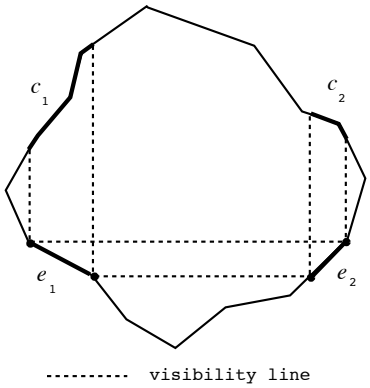


Figure 10: Bottom-Up Sweep for 3 and 4-Contact Cases

for this (e_1, e_2) pair. If an edge e_3 for an upper event on c_1 has a corresponding edge e_4 on c_2, then we solve the 4-contact problem for $\{e_1, e_2, e_3, e_4\}$. Otherwise, we solve the 3-contact problem for $\{e_1, e_2, e_3\}$ *if* e_3 is below c_2. (If e_3 is above c_2 the 3-contact MAAPR is guaranteed to intersect c_2; if it is below it will not intersect.) Note that as the bottom sweepline moves up, the top sweepline moves down. The top-down sweep is similar.

No sorting is required for the sweeps because the xy-monotone chains are naturally ordered in x and y. Because the visibility maps can be precomputed in linear time, c_1 and c_2 are found in constant time. For the bottom-up sweep, the total number of upper events is linear because each edge fragment is visited once for each sweep. A constant amount of work is done at each upper event, since the visibility map is precomputed and solving each 3 or 4-contact problem requires only constant time by Theorem 2.5. Thus, the MAAPR is found in $\Theta(n)$ time. ■

Lemma 5.4 *The 2-contact MAAPR of an n-vertex orthogonally convex polygon can be found in $O(n\alpha(n))$ time.*

Proof: The results of Section 3.3 show that for the 2-contact case we can reduce the MAAPR problem for each pair of diagonally opposite xy-monotone chains in linear time to the problem of finding the maximum in a totally monotone rising double staircase matrix. By Lemma 4.4 we can find the maximum in this matrix in $O(n\alpha(n))$ time. ■

■

5.3 Horizontally and Vertically Convex Polygons

Theorem 5.5 *The MAAPR in an n-vertex horizontally (or vertically) convex polygon can be found in $O(n\alpha(n)\log n)$ time.*

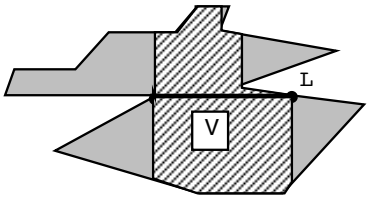


Figure 11: Orthogonally Convex Polygon at Merge Step

Proof: We treat the horizontally convex case, w.l.o.g.. We use a divide-and-conquer algorithm which, at each step, partitions the vertex set using a horizontal line L into two sets of roughly equal size. We determine the endpoints of L in linear time by examining all the edges of the polygon. We recursively find the MAAPR above L and the MAAPR below L. We construct the polygon above L and the polygon below L in linear time by walking around the boundary of P. The merge step requires that we find the MAAPR R_L intersecting L. Let V be the largest polygon containing L that is monotone with respect to L (see Figure 11).

Claim 5.6 *$R_L \subseteq V$. Furthermore, V is orthogonally convex, can be constructed in $\Theta(n)$ time, and has $O(n)$ vertices.*

Proof: Any axis-parallel rectangle R intersecting L must be such that each point $p \in R$ is vertically visible to L; hence $R_L \in V$. To construct V we first supplement the vertices of P with the extra points obtained from the precomputed vertical visibility map. We then claim that two simple traversals of P suffice to construct V. Let l be the left endpoint of L, and r the right endpoint. The first traversal is counterclockwise from l to r to construct the bottom portion of V; the second clockwise from l to r to build the top part. We begin the counterclockwise traversal by following the downward projection of l until it hits the boundary of P. Then we follow the boundary of P unless we encounter either 1) a reflex extreme vertex that is supported from the right by a vertical line, 2) a vertex which is the bottom endpoint of a vertical visibility line emanating from a reflex

extreme vertex that is supported from the left by a vertical line, or 3) the x value of r. In case (1), we follow the visibility line downwards to the boundary of P. In case (2) we follow it upwards to the associated reflex extreme vertex. In case (3), we proceed to r, and terminate the traversal. Constructing the top part of V is similar.

The visibility map introduces at most one new vertex for each vertex of P, so V has $O(n)$ vertices. We visit each vertex at most once during each sweep, so the algorithm requires $\Theta(n)$ time. To show V is orthogonally convex, let b and t be the lowest and highest points (respectively) on P that are visible to L_i. The counterclockwise sweep builds an xy-monotone path from l to b and from b to r. Similarly, the clockwise sweep builds an xy-monotone path from l to t and from t to r. Since the result is a polygon consisting of four xy-monotone chains, such that $l_x \leq b_x \leq r_x$ and $l_x \leq t_x \leq r_x$, it is orthogonally convex. ■

Now we find the MAAPR in V, which, by Theorem 5.2, can be done in $O(n\alpha(n))$ time.

The algorithm therefore yields a recurrence of the form $T(n) = 2T(n/2) + O(n\alpha(n))$, which gives an $O(n\alpha(n)\log n)$ algorithm for finding the MAAPR. ■

5.4 Vertically Separated, Horizontally Convex Polygon

Theorem 5.7 *The MAAPR in an n-vertex vertically separated, horizontally convex polygon (or horizontally separated vertically convex polygon) can be found in $O(n \log n)$ time.*

Proof: We treat the vertically separated, horizontally convex polygon, w.l.o.g.. The results of Section 3.4 show that the 2-contact MAAPR can be found in $O(n \log n)$ time. The remaining cases involve either 3 or 4 contacts. We claim these cases can be solved by an $O(n \log n)$ time divide-and-conquer algorithm which uses the method of Section 5.3 but, in its merge step, solves only the 3 and 4-contact cases for the orthogonally convex polygon V, which from the results of Section 5.2 can be done in linear time.

Claim 5.8 *If the MAAPR in P is a 3 or 4-contact MAAPR, it is also a 3 or 4-contact MAAPR in V.*

Proof: If a rectangle r has at least three contacts with P, it has at least three contacts in V. ■

■

Note that it is also possible to solve the 3 and 4-contact case in $O(n)$ time using a sweep-line algorithm, but that does not improve the overall running time.

5.5 General Polygons with Holes

Theorem 5.9 *The MAAPR in an n-vertex general polygon can be found in $O(n \log^2 n)$ time.*

Proof: We use a divide-and-conquer algorithm which, at each step, partitions the vertices of P using a vertical line L into two sets of roughly equal size . Suppose L is partitioned into k pieces $L_1, L_2, \ldots, L_k$ by the polygon. We can determine the endpoints of L_i, $1 \leq i \leq k$, in linear time by examining all the edges of the polygon. For L_i, we recursively find the MAAPR on the left of L and the MAAPR on the right of L. The merge step requires that we find the MAAPR R_i intersecting L_i. Let H_i be the largest polygon containing L_i that is monotone with respect to L_i.

Claim 5.10 *$R_i \subseteq H_i$. Furthermore, H_i is a vertically separated, horizontally convex polygon, can be constructed in $\Theta(n)$ time, and has $O(n)$ vertices.*

Proof: The proof is similar to the proof of Claim 5.6. We use the horizontal visibility map and two traversals to construct H_i. The relevant reflex extreme vertices are supported from above and below by horizontal lines. H_i is a vertically separated, horizontally convex polygon because each traversal builds a chain that is monotone with respect to the vertical line L_i. ■

By Theorem 5.5, we can find the MAAPR in a vertically separated horizontally convex polygon in $O(n \log n)$ time. It remains to show the following:

Claim 5.11 $\sum_{i=1}^{k} O(n_i \log n_i) = O(n \log n)$.

Proof: Because $\log n_i \leq \log n$, it suffices to show $\sum_{i=1}^{k} \mathrm{O}(n_i) = \mathrm{O}(n)$. The horizontal visibility map on P produces a set of trapezoids T. We can associate $T_i \subseteq T$ with each H_i. Now, $O(n_i) \in O(|T_i|)$. Since each trapezoid in T is associated with at most one i, $\sum_{i=1}^{k} \mathrm{O}(|T_i|) \in \mathrm{O}(|T|) \in \mathrm{O}(n)$. ■

This yields the following recurrence: $T(n) = 2T(n/2) + \mathrm{O}(n \log n)$, which gives $\mathrm{O}(n \log^2 n)$ time for the combined algorithm. ■

6 Lower Bounds

Here we establish lower bounds of time in $\Omega(n \log n)$ for finding the MAAPR in both self-intersecting polygons and general polygons with holes. The latter result gives us both a lower bound of $\Omega(n \log n)$ and an upper bound of $\mathrm{O}(n \log^2 n)$ for general polygons with holes.

These lower bounds contrast with the $\Theta(n)$ time result achievable for the corresponding enclosure problems [13].

6.1 Self-Intersecting Polygons

We prove a lower bound of time in $\Omega(n \log n)$ for finding the MAAPR in a self-intersecting polygon.

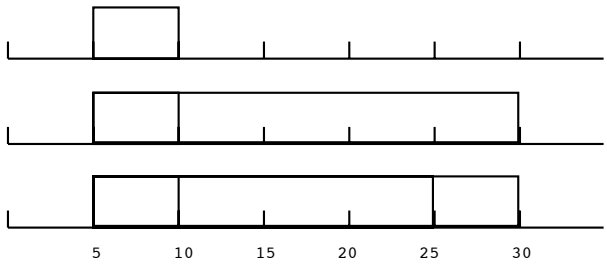


Figure 12: Rectilinear self-intersecting polygon constructed for input = 10, 5, 30, 25

[13]It is interesting to note that the dual problems of largest empty circle and smallest enclosing circle for a set of points also have different lower bounds. The largest empty circle can be constructed in $\Theta(n \log n)$ time, and the smallest enclosing circle can be found in $\Theta(n)$ time [18].

Theorem 6.1 *Finding the MAAPR in an n-vertex self-intersecting polygon requires time in $\Omega(n \log n)$ in both the linear and algebraic decision tree models.*

Proof: We reduce the MAX-GAP problem[14] [5] to the MAAPR problem for self-intersecting rectilinear polygons. Consider an instance of MAX-GAP: given a set of n real numbers $x_1, x_2, ..., x_n$, we must find the maximum difference between two consecutive numbers in the sorted list . We construct from this set, in linear time, a self-intersecting rectilinear polygon of unit height as follows: each x_i in the sequence corresponds to a rectangle $r_i = [(x_1, 0), (x_1, 1), (x_i, 1), (x_i, 0)]$. We start the construction from $(x_1, 0)$, and complete the degenerate rectangle r_1, then constructing $r_2, \ldots, r_n$ (as shown in Figure 12). This construction results in a self-intersecting polygon, with the property that the area of the MAAPR included in it is the solution to the corresponding MAX-GAP problem, thus proving the theorem. ■

6.2 General Polygons with Holes

McKenna et. al. [13] have given a lower bound of time in $\Omega(n \log n)$ for finding the MAAPR in a general polygon with degenerate (zero area) holes. Aggarwal and Suri [3] have given the same lower bound for LER. Using symbolic perturbation [9, 21], both of these can be extended to lower bounds on the computation of the MAAPR in a general polygon. McKenna et. al. use a reduction from the *even distribution problem*: given a set of n real numbers $x_1, x_2, x_3, \ldots, x_n$ (not sorted), determine if there exist adjacent x_i and x_j in the sorted list such that $x_j - x_i > 1$. Their reduction involves the construction of a long horizontal rectangle with vertical "slits" at each x_i. These slits can be thought of as degenerate rectangular holes. Given a slit $(x_i, y_b)(x_i, y_t)$ we can "expand" it to a rectangle with diagonal $(x_i, y_b)(x_i + \epsilon, y_t)$ where $\epsilon > 0$. Of course, if we choose ϵ greater than the value of the minimum gap between points (possibly another $\Omega(n \log n)$ problem), then neighboring slits will overlap and the polygon will be self-intersecting; in effect, we have to know the minimum gap in order to compute the maximum gap.

Symbolic perturbation rescues us from this chicken and egg problem by allowing ϵ to remain un-

[14] In both the linear and algebraic decision tree models (if not enhanced to include floor and ceiling functions), MAX-GAP has a lower bound of $\Omega(n \log n)$.

evaluated until after we have run the MAAPR algorithm. Given an algorithm for computing the MAAPR of a polygon with non-degenerate holes, we modify the way the algorithm evaluates and tests the sign of arithmetic expressions. Since some of the inputs involve ϵ, the arithmetic expressions of the modified algorithm are polynomials in ϵ. For these, the modified algorithm computes the sign by taking the sign of the first (lowest degree in ϵ), non-zero coefficient. We observe that:

- there exists a value of ϵ such that the signs computed by the modified algorithm equal the signs computed by the unmodified algorithm on this value of ϵ (this is the basic theory of symbolic perturbation);
- the running time of the modified algorithm is a constant times the running time of the unmodified algorithm on that value of ϵ.

Hence, any algorithm for the MAAPR in a general polygon can be used to test even distribution via a linear time reduction. Hence the construction of the MAAPR in a general polygon has an $\Omega(n \log n)$ lower bound. We could have also reduced the LER problem to the MAAPR problem by replacing every point in the LER instance by a square of size ϵ.

7 Applications

When a polygon is nearly rectangular, the MAAPR provides a good inner approximation (see Figure 13). Many MAAPR applications have surfaced in our automatic marker-making project for

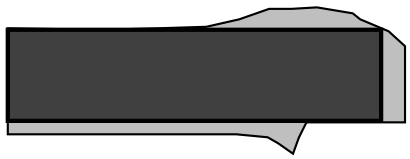
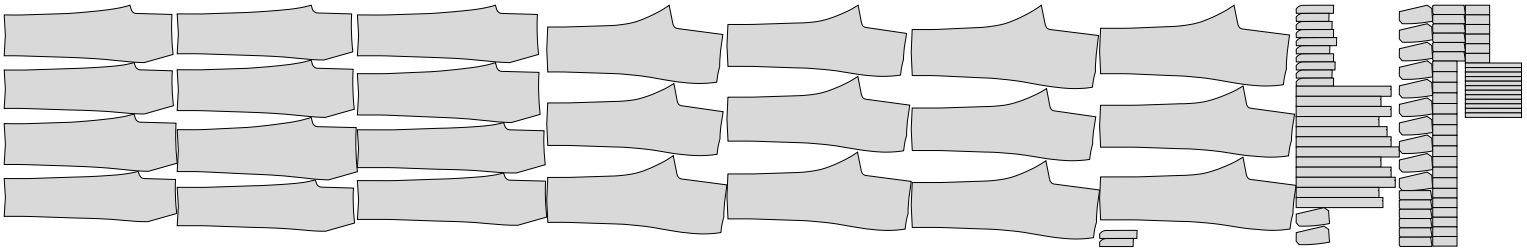


Figure 13: Inner Approximation

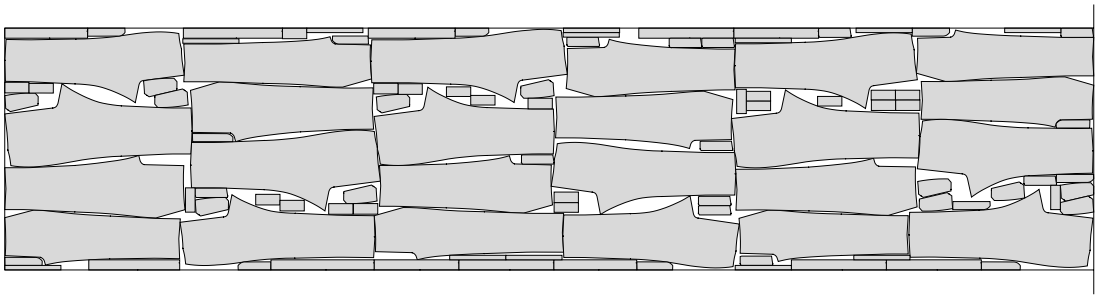
the apparel industry. We briefly describe two of them in this section. The goal of our marker-making project is to automate the task of laying out apparel pattern pieces on a rectangular strip of cloth while minimizing cloth waste [14, 15]. Figure 14 shows an example of the marker-making

task for pants, where the large pieces are *panels* and the small pieces placed among the panels are *trim* pieces.



Set of polygonal parts

Strip of fixed width and unknown length



Name: 37457c
Width: 59.75 in
Length: 269.04 in
Pieces: 108
Efficiency: 89.54%

Packing with 180 degree rotations and xy-flips allowed

Figure 14: The Marker Making Problem

The first application is related to a global layout strategy we are experimenting with. This strategy obtains MAAPRs of *gaps* (holes between adjacent panels) and then solves the problem of packing the nearly rectangular trim pieces into the MAAPRs using techniques from the rectangle packing domain (see Figure 15).

Obtaining the appropriate set of gaps is a challenging problem in its own right, and has many uses within our automatic marker-making system beyond the global strategy mentioned above. This

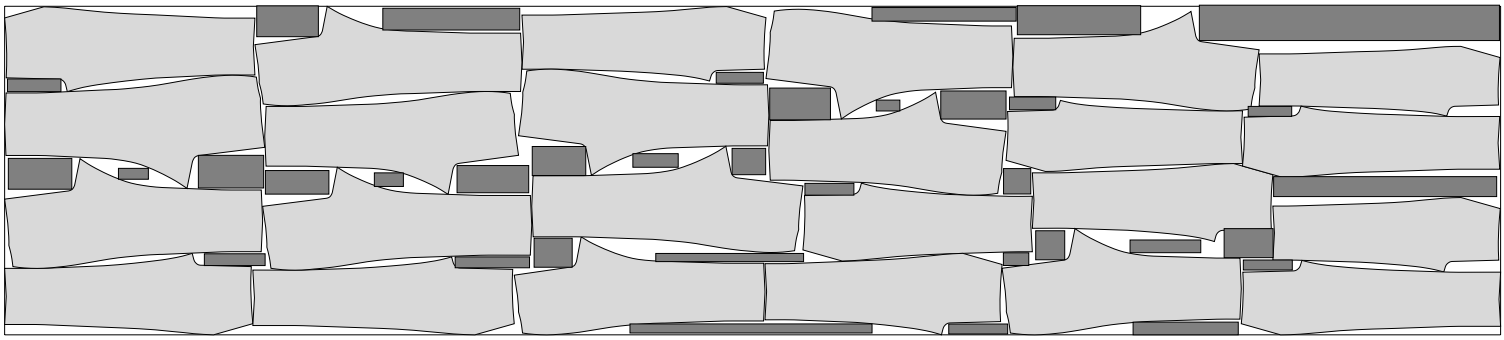


Figure 15: Pants Marker with Selected Gap MAAPRs

second application uses MAAPRs in an unusual way to construct gaps. Of course, one could apply a union algorithm to obtain the gaps. However, the result might be one gap with many narrow "necks" which snakes around among the panels. This occurs frequently in practice. The MAAPR of such a gap is not useful. We want to partition the gaps to eliminate narrow necks.

Ideally, the set of gaps we seek is the set of all points which can be "inked" by translations of our set of trim pieces $t_1, t_2, ..., t_n$. This set of gaps can be expressed as the union of a set of polygons as follows. Let v_i be the *valid-area polygon* for t_i with respect to the set of panels. The polygon v_i is obtained using the Minkowski difference operator, and it represents all the positions for the local origin of t_i such that t_i will not overlap any panel piece. We find the MAAPR of each t_i and then find the largest rectangle r that fits inside all those MAAPRs. Let v_r be the valid-area polygon of r. The key observation is that all of the connected components of the v_i's are contained within the connected components of v_r[15]. Because of this fact, we can use each connected component of v_r as a "gap seed". For each v_r we dilate each v_i by its trim piece, using the Minkowski sum. The union of the resulting polygons is the set of all points which can be inked by our trim pieces.

8 Open Problems and Future Work

For the general polygon with holes, we establish a lower time bound of $\Omega(n \log n)$ and upper bound of $O(n \log^2 n)$. This leaves room for improvement by a factor of $\log n$. Our running time for xy-

[15]To enforce this, one must use offsets to compensate for the difference between the local origin of r and the local origin of each trim piece.

monotone polygons is optimal, but for orthogonally convex polygons, horizontally convex polygons, and vertically separated, horizontally convex polygons, no non-trivial lower bound has been established. It might still be possible, therefore, to improve the running time for those types of polygons.

In many applications, a fast approximate MAAPR algorithm would be valuable.

9 Conclusion

Our general framework for solving the 2-contact case allows us to match the best known running times for the rectilinear counterparts of each of the five non-rectilinear polygons we examine. There may be other classes of polygons that are amenable to this general method.

Acknowledgments

The authors gratefully acknowledge the helpful comments made by Pankaj Agarwal, Zhenyu Li, Marios Mavronicolas, and Binhai Zhu. We also acknowledge the background information provided by David Dobkin and Joseph O'Rourke, and the unpublished manuscript provided by Alok Aggarwal.

References

[1] A. Aggarwal and M. M. Klawe. Applications of Generalized Matrix Searching to Geometric Algorithms. *Discrete Applied Mathematics*, 27:3–23, 1990.

[2] A. Aggarwal, M.M. Klawe, S. Moran, P. Shor, and R. Wilber. Geometric applications of a matrix-searching algorithm. *Algorithmica*, 2:195–208, 1987.

[3] A. Aggarwal and S. Suri. Fast Algorithms for Computing the Largest Empty Rectangle. In *Proceedings of the 3rd ACM Symposium on Computational Geometry*, pages 278 – 290, 1987.

[4] A. Aggarwal and S. Suri. Fast Algorithms for Computing the Largest Empty Rectangle. *Personal communication*, 1992.

[5] A. Aggarwal and J. Wein. *Computational Geometry Lecture Notes for MIT 18.409.* 1988.

[6] N. Amenta. Largest Volume Box is Convex Programming. *Personal communication*, 1992.

[7] B. Chazelle. Triangulating a Simple Polygon in Linear Time. In *Proceedings of the Thirty-First Annual Symposium on Foundations of Computer Science*, pages 220–230, 1990.

[8] B. Chazelle, R.L. Drysdale III, and D.T. Lee. Computing the largest empty rectangle. *SIAM J. Comput.*, 15:300–315, 1986.

[9] H. Edelsbrunner and E.P. Mücke. Simulation of Simplicity: A technique to cope with degenerate cases in geometric algorithms. *ACM Transactions on Graphics*, 9:66–104, 1990.

[10] J.S.Chang and C.K. Yap. A Polynomial Solution for the Potato-peeling Problem. *Discrete Computational Geometry*, 1:155–182, 1986.

[11] M. M. Klawe. Superlinear Bounds on Matrix Searching. In *Proceedings of the First ACM Symposium on Discrete Algorithms*, pages 485–493, 1990.

[12] M. M. Klawe and D. J. Kleitman. An Almost Linear Time Algorithm for Generalized Matrix Searching. *SIAM Journal of Discrete Mathematics*, 3(1):81–97, 1990.

[13] M. McKenna, J. O'Rourke, and S. Suri. Finding the largest rectangle in an orthogonal polygon. In *Proceedings of the 23rd Allerton Conference on Communication, Control, and Computing*, pages 486–495, 1985.

[14] V. Milenkovic, K. Daniels, and Z. Li. Automatic Marker Making. In *Proceedings of the Third Canadian Conference on Computational Geometry*, 1991.

[15] V. Milenkovic, K. Daniels, and Z. Li. Placement and Compaction of Nonconvex Polygons for Clothing Manufacture. In *Proceedings of the Fourth Canadian Conference on Computational Geometry*, 1992.

[16] A. Naamad, W.L. Hsu, and D.T. Lee. On the maximum empty rectangle problem. *Discrete Appl. Math.*, 8:267–277, 1984.

[17] M.H. Overmars and D. Wood. On rectangular visibility. *J. Algorithms*, 9:372–390, 1988.

[18] F. Preparata and M. Shamos. *Computational Geometry: An Introduction.* Springer-Verlag, New York, 1985.

[19] S. Schuierer, G.J. E. Rawlins, and D. Wood. A generalization of staircase visibility. In *Proceedings of the 3rd Canadian Conf. Comput. Geom.*, pages 96–99, 1991.

[20] D. Wood and C. K. Yap. The orthogonal convex skull problem. *Discrete Computational Geometry*, 3:349–365, 1988.

[21] C.K. Yap. A geometric consistency theorem for a symbolic perturbation scheme. *Journal of Computer and System Sciences*, 40:2–18, 1990.